University of Petroleum and Energy Studies, Dehradun

Internship - Low Level Design

on

Digital Forensics on a Compromised System

Team members:
Jai Maniktalla – 590010183
Anandshankar Iyer – 590010637
Shashwat Tripathi – 590011321

Guided by: Dr. Sumitra

Industry Mentor: Mr. Mohsin Quresh

Table of Contents

1. Introduction.....6
   1.1. Scope of the document.....6
   1.2. Intended audience.....6
   1.3. System overview.....6

2. Low Level System Design.....7
   1.1. Sequence Diagram.....7
   1.2. Navigation Flow/UI Implementation.....7
   1.3. Screen Validations, Defaults and Attributes.....7
   1.4. Client-Side Validation Implementation.....8
   1.5. Server-Side Validation Implementation.....8
   1.6. Components Design Implementation.....8
   1.7. Configurations/Settings.....9
   1.8. Interfaces to other components.....9

3. Data Design.....10
   1.1. List of Key Schemas/Tables in database..... 10
   1.2. Details of access levels on key tables in scope..... 10
   1.3. Key design considerations in data design..... 10

4. Details of other frameworks being used.....11
   1.1. Session Management..... 11
   1.2. Caching..... 11

5. Unit Testing.....12

6. Key notes..... 13

7. Reference..... 13

1. Introduction

1.1 Scope of the Document

This document defines the detailed low-level design and implementation strategy for a digital forensics investigation system focused on detecting and analyzing memory-resident supply chain attacks (e.g., CVE-2024-3094).

The scope includes:

- Design of a sandboxed forensic lab environment (Proxmox + pfSense)
- Implementation of threat emulation and memory acquisition workflows
- Analysis of volatile memory artifacts and network telemetry
- Integration of forensic tools like Volatility 3 and Wireshark

This document does not cover high-level research theory or literature explanations but focuses on technical execution and system design.

1.2 Intended Audience

This document is intended for:

- SOC Analysts and Threat Hunters
- Digital Forensics Investigators (DFIR teams)
- Cybersecurity Researchers
- System Architects implementing forensic labs
- Academic evaluators reviewing system design

1.3 System Overview

The system is a controlled forensic investigation environment designed to:

- Simulate a supply chain attack (XZ Utils backdoor)
- Capture live memory and network traffic
- Analyze artifacts to reconstruct attack behavior

Core components:

- Proxmox Hypervisor → hosts entire lab
- pfSense Firewall → network segmentation + logging
- Kali Linux (Attacker VM) → executes exploit
- Victim Linux VM → compromised system
- Volatility 3 + Wireshark → forensic analysis tools

(see System Architecture Diagram – Appendix A)

2. Low Level System Design

2.1 Sequence Diagram

Flow of execution:

1. Attacker VM generates cryptographic payload
2. Payload sent via SSH handshake to victim
3. Malicious liblzma hooks RSA_public_decrypt
4. RCE triggered in memory (no disk traces)
5. pfSense captures network traffic (.pcap)
6. Hypervisor extracts RAM dump
7. Volatility analyzes memory artifacts
8. Correlation engine builds attack timeline

(see Sequence Diagram – Appendix B)

2.2 Navigation Flow / UI Implementation

System interaction flow:

- **Admin Dashboard**
  - Start/Stop VMs
  - Trigger exploit
  - Capture memory
- **Monitoring Panel**
  - Network logs (pfSense)
  - SSH traffic visualization
- **Forensic Analysis Interface**
  - Upload RAM dumps
  - Run Volatility plugins
- **Reporting Module**
  - Generate timeline
  - Export findings

2.3 Screen Validations, Defaults and Attributes

Field	Validation	Default
VM Status	Must be running	Stopped
Memory Dump Path	Valid file	/dumps
Network Capture	Enabled/Disabled	Enabled
Kernel Profile	Must match OS	Required

2.4 Client-Side Validation Implementation

- Input validation for:
  - File uploads (.raw, .mem)
  - VM configuration fields
- Prevent invalid actions:
  - Memory capture if VM is off
- UI alerts:
  - Missing kernel symbols
  - Invalid file formats

2.5 Server-Side Validation Implementation

- Validate:
  - Integrity of memory dumps (hash check)
  - Correct kernel profile for Volatility
- Authentication:
  - Role-based access (Admin / Analyst)
- Logging:
  - All actions recorded for audit

2.6 Components Design Implementation

Core Modules:

1. **Virtualization Layer**:
   - Proxmox VE managing VMs
2. **Network Layer**:
   - pfSense for VLAN segmentation
3. **Attack Engine**:
   - PoC scripts for CVE-2024-3094
4. **Forensic Engine**:
   - Volatility 3 for memory parsing
5. **Analysis Engine**:
   - Correlates memory + network

(see Component Diagram – Appendix C)

2.7 Configurations / Settings

- VLAN segmentation:
  - Attacker subnet
  - Victim subnet
- Logging:
  - Full packet capture enabled
- Kernel profiling:

  - System.map + DWARF required
- Resource allocation:
  - RAM: 8GB+
  - Storage: High-speed NVMe

(see Deployment Diagram – Appendix E)

2.8 Interfaces to Other Components

- **Proxmox ↔ pfSense** → network routing
- **pfSense ↔ Wireshark** → packet capture
- **VM ↔ Volatility** → memory analysis
- **Attack scripts ↔ Victim VM** → exploit delivery

3. Data Design

3.1 List of Key Schemas / Tables

Table Name	Description
VM_Config	VM details and state
Memory_Dumps	RAM dump metadata
Network_Logs	Captured packets
Analysis_Results	Forensic outputs
Users	Access control

(see Data Flow Diagram – Appendix D)

3.2 Details of Access Levels on Key Tables

Role	Access
Admin	Full access
Analyst	Read + Analyze
Viewer	Read-only

3.3 Key Design Considerations in Data Design

- High storage for large RAM dumps
- Efficient indexing for logs
- Secure storage of forensic artifacts
- Data integrity using hashing

4. Details of Other Frameworks Being Used

4.1 Session Management

- Token-based authentication
- Session timeout for security
- Logging of user actions

4.2 Caching

- Cache frequent analysis results
- Reduce repeated Volatility execution
- Improve UI performance

5. Unit Testing

- Validate:
  - Memory capture functionality
  - Network packet logging
  - Volatility plugin outputs
- Test cases:
  - Successful RCE detection
  - Failed exploit scenarios
- Automation:
  - Script-based validation

6. Key Notes

- Entire system runs in **isolated environment (zero-trust)**
- Focus on **volatile memory forensics**
- Avoid system reboot (loss of evidence)
- Ensure **kernel compatibility for analysis tools**
- Maintain strict **ethical hacking boundaries**

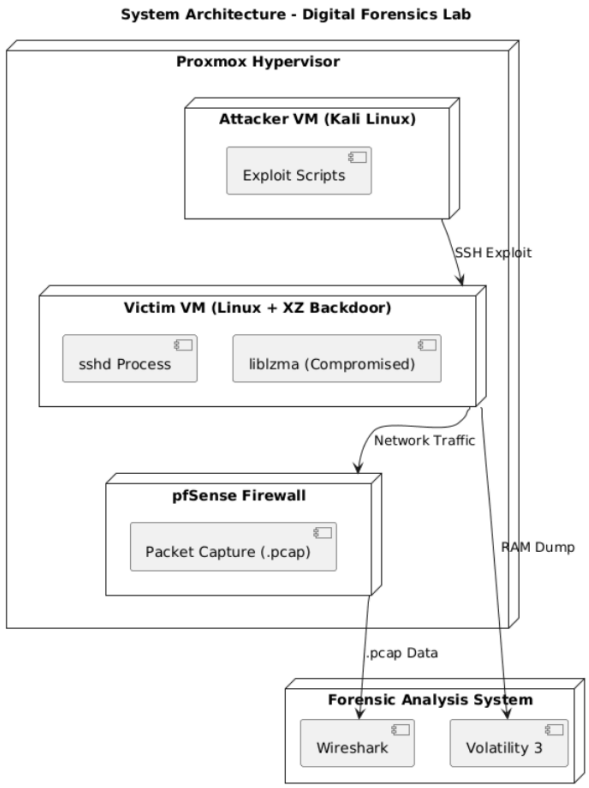
7. Reference

Based on:

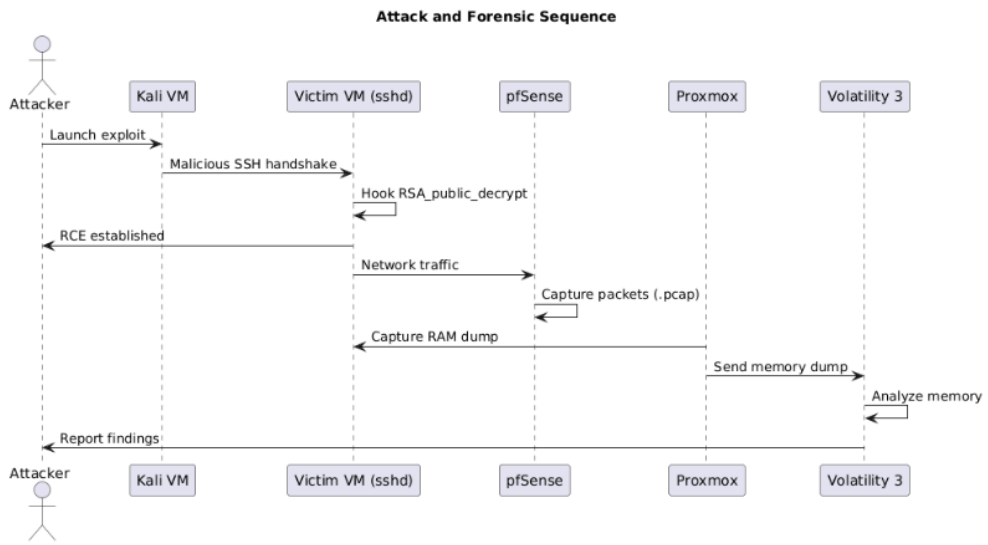
- Digital Forensics Investigation of Compromised System
- Volatility 3 documentation
- MITRE ATT&CK framework
- CVE-2024-3094 research papers

Appendix

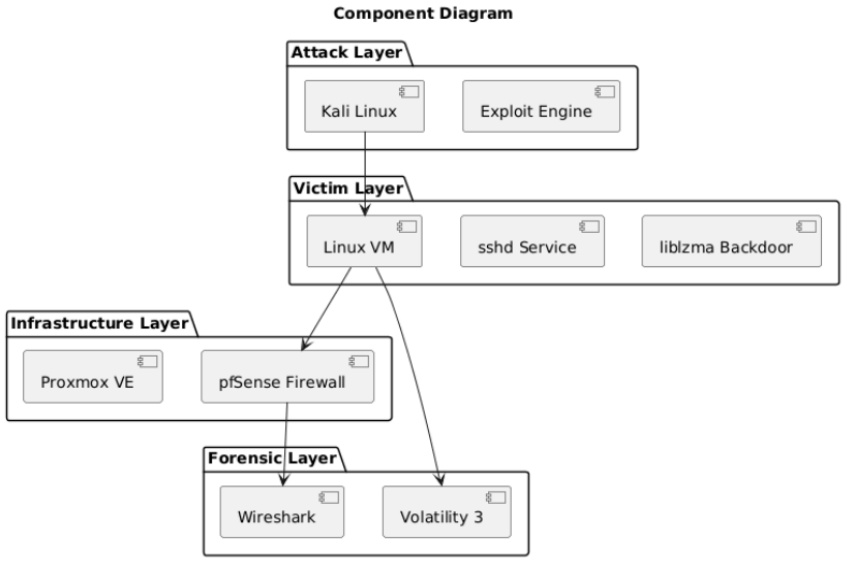
Appendix A:



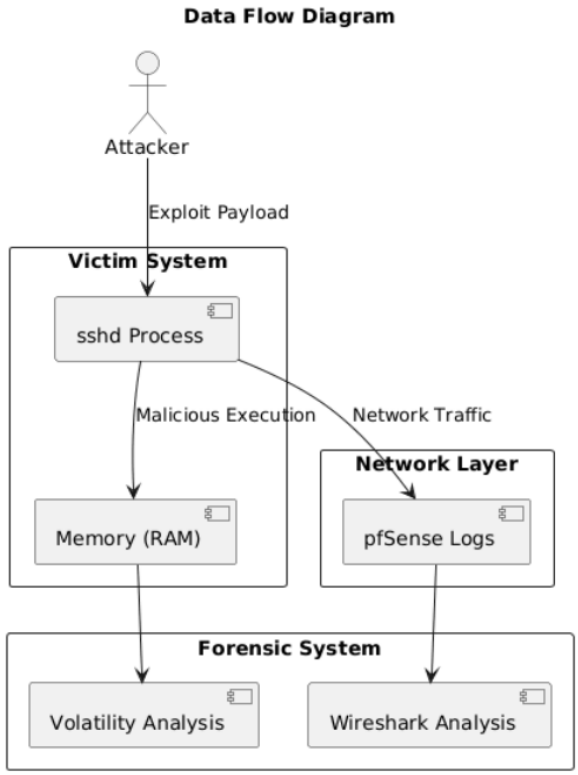
Appendix B:



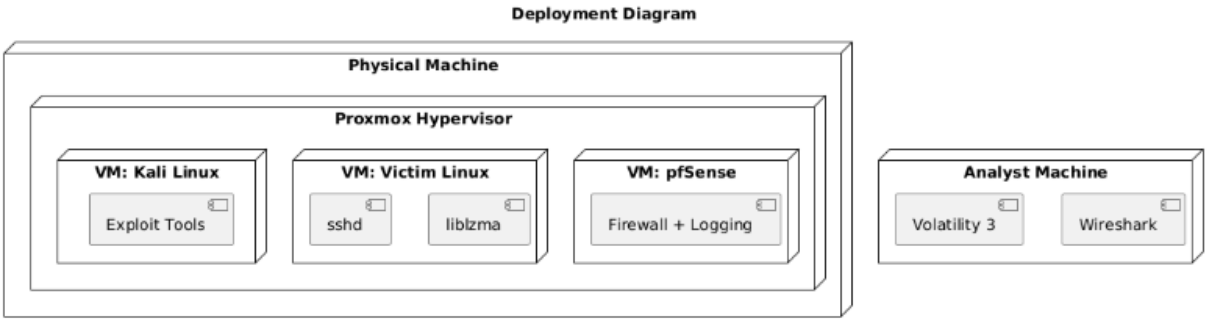
Appendix C:



Appendix D:



Appendix E:


Deployment Diagram
Physical Machine
Proxmox Hypervisor
VM: Kali Linux
Exploit Tools
VM: Victim Linux
sshd
liblzma
VM: pfSense
Firewall + Logging
Analyst Machine
Volatility 3
Wireshark